BUILDBRIEF

Product Strategy, System Architecture & Agent Implementation Plan

31 October 2026 | Strategy v1.1

Strategic decisions locked

- Both interaction modes: inside BuildBrief and alongside existing coding agents.
- Hybrid execution: local-first with optional cloud services.
- Both markets: founders and developers, with role-adapted experiences.

From intent to proof, and from proof to understanding.

This is a strategic and engineering plan, not a claim that all proposed capabilities are already implemented. The coding agent must inspect the live repository before changing code.

0. Executive decision

BuildBrief should become a **cross-agent software-development control layer**. It helps a person turn an idea into an owner-approved specification, pass that intent and relevant codebase context to a coding agent, observe supported work, compare the result with approved intent, route corrections back to the agent, and help people understand what changed.

Confirmed strategic decisions

• Interaction: **BOTH**. Work inside BuildBrief or alongside an existing coding agent.

• Execution: **HYBRID**. Local-first for repository access, private source and deterministic checks; optional cloud services for sync, collaboration and hosted work.

• Market: **BOTH**. Founders and developers share one evidence engine but receive role-adapted experiences.

Positioning: “The control layer between what you intend, what your coding agent changes, and what you can prove.” Short public line: “Build with AI. Stay in control.”

Do not become another IDE, generic prompt generator, generic spec writer or code-review dashboard. The value is continuity of an approved intent contract across clarification, agent execution, evidence, correction and human understanding.

1. Product thesis and invariants

Core workflow: Idea → Clarify → Approve → Build → Observe → Verify → Correct → Understand.

• Capture the raw idea and preserve it.

• Identify ambiguity, missing decisions, constraints and contradictions.

• Ask a small number of adaptive questions with recommended answers and trade-offs.

• Produce a structured, versioned intent contract for owner approval.

• Inspect the connected repository and select relevant context.

• Generate an implementation brief and route it to the selected agent.

• Observe supported agent events, diffs, tests and preview state.

• Compare evidence with approved criteria; expose gaps and uncertainty.

• Let the owner clarify, accept, dismiss or approve a correction.

• Route approved corrections to the agent and re-check the changed revision.

• Explain the result in founder, developer, QA or other role language.

Four invariants

• Intent is not approved truth until an authorised owner approves it. AI extraction remains a draft.

• Source code is not runtime proof. Source, test, staging and production evidence are distinct.

• A prompt is not a patch. A fix is only verified after a new revision and fresh checks.

• A dashboard score is not comprehension. Learning claims require consented assessments and a defined rubric.

2. Users, market and differentiation

Founder / product owner: describes outcomes without knowing every technical decision; needs guided clarification, sensible defaults, trade-offs, decisions and plain-English status.

Developer / technical lead: needs precise acceptance criteria, repository-aware context, revision-pinned evidence, tests, diffs, provenance, adapter controls and scoped correction packs.

QA, marketing, support and operations: need role-specific explanations of behaviour, limitations, changed features and release implications, without unnecessary access to secrets or source.

Differentiation hypothesis: owner-controlled versioned intent + cross-agent adapters + revision-specific evidence and honest uncertainty + reverse change triage + role-based knowledge transfer. This is a hypothesis to validate, not a proven moat.

Competitive reality: GitHub Spec Kit already provides structured specification, planning, implementation and convergence workflows. Kiro documents specs, steering, hooks, MCP and permissions. Codex offers SDK and App Server integration paths.

Do not claim competitors lack these capabilities. BuildBrief must win through the tested combination and user experience.

3. Project Room UX

Build a **Project Room**, not an admin dashboard full of raw logs.

Primary navigation

• Home / Projects: active projects, pending decisions, stale verification and next action.

• Project Room: project status, active task, approved intent, connected agent, changes and preview.

• Build Brief: requirements, questions, assumptions, decisions, acceptance criteria and versions.

• Activity: agent timeline, tool events, file changes, commands and tests.

• Evidence: criterion-by-criterion status, source references, test runs, runtime observations and evidence grade.

• Corrections: proposed action, generated agent prompt, approval, implementation and re-check.

• Understand: role explanations, evidence-backed Q&A, onboarding and comprehension exercises.

• Connections & Settings: adapters, cloud sync, permissions, privacy and usage budgets.

Design direction: professional, restrained, confident, calm. Strong typography, generous spacing, clear hierarchy, polished empty/loading/error states. Use motion to explain state changes, not as decoration. Use 2D diagrams for requirement-to-file/test relationships; 3D is optional product storytelling, not core navigation. Respect reduced motion and accessibility. Show plain-language summaries first, technical detail on expansion. Never show internal enums, event IDs, raw hashes or `partial=1` as primary copy.

Two modes, one truth: Founder mode shows outcomes, decisions and plain-English explanations. Developer mode shows criteria, files, diffs, tests and provenance. Both read the same approved intent and evidence.

4. User journeys

New idea: user describes an AI invoice app → chooses new or existing repository → BuildBrief identifies missing decisions (users, invoice lifecycle, currencies, tax, payment, permissions, data retention, integrations) → asks high-value questions → recommends defaults with reasons and trade-offs → user accepts, edits or defers → system creates draft requirements, assumptions, out-of-scope list and acceptance tests → owner approves version → brief is routed to agent.

Existing repository: connect local or explicitly selected cloud repo → deterministic inspection detects language, framework, tests, entry points and dirty state → show inspected scope and exclusions → user selects or describes a feature → retrieve only relevant context → produce draft brief → owner approves.

Alongside agent: user keeps using Codex/OpenCode/Muse Code/Claude Code → supported hook, MCP, CLI or manual import captures prompt/session → capture completeness is visible → owner confirms extracted requirements → supported events and changes are linked → verify current revision.

Inside BuildBrief: user starts task in Project Room → BuildBrief starts/resumes supported agent runtime → streams timeline and approvals where adapter supports them → user steers or interrupts where supported → session and event provenance are stored.

Correction loop: show unmet/contradicted criterion or possible unrequested diff → show evidence, grade and revision → distinguish requirement ambiguity from implementation failure → owner clarifies, accepts, dismisses or approves correction → generate scoped correction pack → send to agent → new revision → fresh checks → immutable new report.

5. Adaptive clarification and prompt intelligence

Do not use a long generic questionnaire. Use an adaptive decision graph:

• Extract candidate requirements and uncertainty.

• Rank unknowns by impact, ambiguity, dependency and cost of changing later.

• Ask the highest-value question first.

• Explain the recommended answer and trade-off.

• Accept custom answer, “not sure”, “decide later” or “not applicable”.

• Stop when enough is known for a useful first release; keep assumptions visible.

Example: **How should customers pay an invoice?** Recommended: one supported payment link provider. Explain reduced manual reconciliation, provider availability and fees. Alternatives: bank transfer instructions, card payments, multiple methods. Allow custom answer or defer.

Specification output: problem, target user, outcomes, functional and non-functional requirements, constraints, assumptions, unresolved questions, out-of-scope items, roles, acceptance criteria, tests, data handling and owner approval/version.

Implementation brief: objective; approved requirements/version; relevant architecture; discovered files/modules and rationale; constraints; acceptance tests; out-of-scope changes; security/privacy rules; progress reporting; completion evidence format. Clearly separate approved requirements, assumptions, recommendations and agent choices.

6. Hybrid architecture and integrations

Local runtime: repository connector, file index, SQLite event store, append-only JSONL export, Git snapshot/diff, evidence resolver, CLI, local web app, local MCP server, local agent connectors, redaction and ignore rules.

Cloud control plane (optional): accounts, workspaces, encrypted sync of approved records, teams/roles, hosted agent sessions where supported, background jobs, notifications, usage metering.

Domain services: intent/approval, clarification, repository context, prompt compiler, agent coordinator, evidence collection, verification, correction/re-check, explanation/learning and evaluation.

Agent adapter contract: discover/install/check connection; create/resume/interrupt; send message/steer; stream events; observe tool/file/command events; request/resolve approvals; capture transcript/session metadata; report capabilities and event gaps. Never assume every agent supports every feature.

Recommended integration order

• Local CLI + MCP baseline.

• OpenCode adapter using documented plugins/events/MCP after version testing.

• Codex adapter: SDK for programmatic local tasks; App Server for deep embedded experience, streamed events and approvals.

• Muse Code hooks/MCP/skills after validating installed version.

• Other agents once the adapter contract is stable.

Alongside first, embedded second. Alongside mode avoids recreating each provider's terminal, editor, runtime and approval system. Embedded mode should use a stable provider SDK/App Server, not a home-grown coding agent.

Hybrid rules: local by default for private source and deterministic inspection; cloud is opt-in and shows exactly what is uploaded; send minimum context; secrets stored in OS credential facility/secret manager, never URLs/source/logs; offline checks remain useful without model access.

7. Data model and evidence contract

Core entities: Project; Intent; immutable IntentVersion; Decision; ApprovalEvent; AgentSession; AgentEvent; RepositorySnapshot; EvidenceItem; VerificationRun; CorrectionPack; LearningActivity.

Evidence levels

• E0: no usable evidence.

• E1: repository structure/symbol discovery.

• E2: source evidence tied to commit/blob and line range.

• E3: test/build/lint evidence tied to reproducible run and revision.

• E4: authorised runtime/staging observation with environment and timestamp.

• E5: production telemetry with defined measurement window.

Store separate dimensions: implementation state (supported/partial/not found/conflicting/unknown); evidence grade; search coverage; freshness; review state. “Not found” is only defensible within declared scope. “Contradicted” requires positive incompatible evidence. Insufficient search means unknown, not missing.

8. Verification, reverse drift and correction

Pipeline: load approved intent version and exact revision → resolve criteria to files/symbols/tests → deterministic Git/AST/config/test/citation checks → model only for unresolved semantic interpretation → validate strict schema → mechanically verify paths/lines/hashes/test IDs → calculate coverage and evidence grade → abstain when evidence is insufficient → persist immutable run with model/prompt version, context manifest, cost, latency and errors.

Reverse review: map changed hunks to approved criteria/tasks. Flag “possibly unrequested changes”, not “unauthorised changes” by default. Allow legitimate refactors, formatting and indirect support to be explained and dispositioned by a human.

Correction pack: criterion; evidence and revision; desired outcome; smallest relevant context; constraints/protected areas; acceptance tests; what must not change; completion report.

Re-check: correction is not complete until a new revision exists and relevant checks rerun. Historical reports remain immutable.

9. Security, privacy and trust

Threats: accidental source/credential upload; prompt injection in repository text; fabricated citations/test claims; unauthorised approvals; path traversal/symlink escape/unsafe scripts; cross-project leakage; stale evidence; missing hooks presented as complete monitoring.

Controls: explicit local/cloud privacy mode; per-project consent; `.buildbriefignore`; secret redaction; no untrusted scripts on cloud host; isolated test runners; canonical path and symlink checks; size/archive limits; repository content treated as data, not privileged instructions; owner-only approval; role checks; encryption in transit/at rest; short-lived access; audit and deletion/export; integration coverage and capture gaps visible.

Trust principle: BuildBrief must be more conservative than the coding agent it monitors. If it cannot prove something, it must say so.

10. API/token economy

Token efficiency is a product requirement. Use Git, AST, symbol search, tests, diff parsing, hashes and schema validation before model calls. Model only unresolved semantic questions.

• Build repository map locally.

• Retrieve only files relevant to approved criteria.

• Exclude secrets, binaries, generated files and unrelated directories.

• Bound input/output tokens and show coverage warnings on truncation.

• Cache stable instructions/schemas and revision-specific context.

• Cache key: project, intent version, repo tree/commit, selected file hashes, test run IDs, model, prompt version and schema.

• Invalidate dependent evidence when inputs change.

• Per-project and per-user caps, per-run estimates, daily limits, retry ceilings and hard provider/account cap where available.

• No paid model calls in routine unit-test CI; use offline fixtures.

• One explicitly approved live smoke test for each new provider path.

• Log tokens, cache use, latency, retries and cost.

• Separate coding-agent subscription credits, BuildBrief inference, infrastructure and engineering labour. Agent connection does not mean free access to that agent's model.

11. Roadmap and release gates

Phase 0 — Stabilise current foundation: inspect actual repo/M4e/M5 status; resolve capture/approve/verify inconsistency; owner-side browser test; safe Git commits; confirm secret/auth boundaries; repeatable demo. Gate: existing evidence-first workflow is reliable.

Phase 1 — UX redesign: Home, Project Room, Build Brief, Activity, Evidence and Corrections; founder/developer modes; plain language; responsive/accessibility. Gate: first-time user understands status and next action without internal logs.

Phase 2 — Alongside-agent MVP: adapter contract, CLI/MCP, first two adapters, event completeness, session identity, changed revision, repository-aware brief. Gate: existing agent use produces traceable intent/evidence.

Phase 3 — Adaptive planning: question graph, recommendations, owner-approved spec, tasks and tests. Gate: founder can reach useful brief without generic form.

Phase 4 — Monitoring and correction: supported event stream, diff mapping, evidence-grade verification, approved correction, fresh re-check. Gate: real end-to-end loop.

Phase 5 — Embedded agent: one stable SDK/App Server integration with approvals, interruption, streaming and recovery. Gate: clear user value without recreating an IDE.

Phase 6 — Cloud/team scale: encrypted sync, teams, GitHub App, hosted jobs, tenant isolation, pricing and support.

Defer native mobile IDE, advanced 3D, autonomous agent swarms, broad enterprise compliance and every possible integration.

12. Agent-ready backlog and acceptance

A. **Repository audit (P0):** inventory repo, branch, dirty state, tests, reports, routes, architecture, failures. Acceptance: working/partial/broken/untested clearly distinguished.

B. **Intent approval model (P0):** immutable versions, raw request, extraction provenance, decisions, actor. Acceptance: AI draft cannot be owner-approved; amendment creates new version; unauthorised approval denied.

C. **Repository/evidence engine (P0):** root validation, ignore rules, Git snapshot, diff, symbol retrieval, citation resolver, freshness. Acceptance: stale SHA, nonexistent line, traversal, symlink, secret and oversized file safely handled.

D. **Adapter contract (P0/P1):** provider-neutral capability negotiation, local CLI/MCP, first two adapters. Acceptance: missing events/capability gaps visible.

E. **Clarification/brief compiler (P0):** question priority, recommendation rationale, custom/deferred answers, assumptions, approved brief. Acceptance: five realistic idea scenarios create useful questions and testable criteria.

F. **Activity/change mapping (P1):** prompts, tool events, files, commands, tests, completion. Acceptance: ordered recoverable timeline with gaps labelled.

G. **Verification/reverse review (P0):** deterministic-first, semantic adjudication, grades, coverage, abstention, drift. Acceptance: independent positive/negative/unknown benchmark; mechanically checked citations.

H. **Correction/re-check (P0):** scoped correction pack, agent handoff, new revision and fresh checks. Acceptance: prompt alone cannot change status.

I. **Project Room UX (P0):** onboarding, role modes, questions, activity, evidence, correction and empty/error states. Acceptance: founder and developer usability test.

J. **Learning (P1):** grounded explanations/quizzes, rubric and consent. Acceptance: invalid citations rejected, stale questions retired, no unmeasured score.

K. **Security/cost (P0):** local/cloud boundary, redaction, budgets, telemetry, caching. Acceptance: no secrets in logs/URLs; safe budget exhaustion; offline mode works.

Definition of done: typed schemas; validation; positive and negative tests; permission checks; timeout/error paths; source/commit provenance; audit event; documentation; clean setup; no fabricated metrics; UI screenshots; recorded test report.

13. Testing and validation

Engineering tests: unit, integration, end-to-end, security and resilience. Include interrupted sessions, rate limits, offline mode, missing hooks, traversal, symlinks, prompt injection, stale evidence and tenant isolation.

Benchmark cases: correct implementation, partial, absent within adequate coverage, contradiction, ambiguous requirement, misleading comments, fake citation, stale evidence, legitimate refactor, deleted test and sensitive edit. Report precision, recall, false-positive rate, abstention, citation validity, coverage, stale detection, latency and cost. Separate dev and holdout sets; independently label ground truth; never hand-edit metrics.

User research: 5–10 founders/builders and 3–5 developers/technical leads. Measure time to approved brief, clarification rounds, time to detect mismatch, useful-alert/nuisance ratio, correction attempts, founder understanding, developer trust and repeat use. Friendly testers do not prove product-market fit.

14. Budget and assumptions

Planning conversion: **US$1 = GH■11.62**, Bank of Ghana interbank mid-rate published 30 October 2026. Refresh before spending; actual bank/card rates differ. Estimates are planning ranges, not vendor quotes. Exclude founder salary, tax, legal fees, enterprise licences and unusual data-transfer costs.

One-time MVP, 6–10 weeks

Category	Lean	Base	Higher assurance
Product/UI design	$300–700	$800–1,500	$2,000–3,500
Engineering	$1,500–3,000	$4,000–7,500	$9,000–15,000
QA/security	$200–500	$700–1,500	$2,000–4,000
Tools/assets	$50–150	$150–400	$500–1,000
Contingency	$205–653	$565–1,635	$1,350–3,525
Total USD	**$2,255–5,003**	**$6,215–12,535**	**$14,850–27,025**
Approx. GHS	**GH■26,200–58,100**	**GH■72,200–145,700**	**GH■172,600–314,000**

Lean assumes founder-led product/engineering and limited paid help. Base assumes some contract design/engineering and external QA. Higher assurance includes deeper security and robust integrations. Replace with quotes and scoped hours.

Monthly operating costs

Category	Lean	Base	Growth
Hosting/database/storage	$20–60	$80–200	$300–800
BuildBrief inference	$10–40	$50–150	$200–700
Monitoring/email/misc	$5–20	$20–60	$80–200
Backup/security/tools	$10–30	$30–100	$100–300
Total USD/month	**$45–150**	**$180–510**	**$680–2,000**
Approx. GHS/month	**GH■523–1,743**	**GH■2,092–5,926**	**GH■7,902–23,240**

Start with a **$25/month inference ceiling** for private alpha, not a promise of a specific call count. Raise to $100 only after cost-per-useful-task is measured. An in-app warning is not a hard cap. Coding-agent credits are separate.

Measure tokens per clarification/verification, retrieved files/bytes, cache hit rate, retries, cost per useful correction and cost per active project. Do not set final subscription prices until 10–20 users demonstrate repeat value and unit costs are known.

15. First 30 days

Week 1: audit repo, repair core flow, browser-test, finalise approval model, map UI, define adapter contract. Do not start a rewrite before this audit.

Week 2: build Project Room, new/connect project, adaptive questions, recommendations, owner review/version history, founder/developer modes.

Week 3: local CLI/MCP, first two adapters, repository context selection, prompt compiler, session identity and event gaps.

Week 4: activity timeline, revision-pinned evidence, verification, correction/re-check, benchmark and one real end-to-end demo.

16. Metrics and stop gates

North-star: for an approved requirement at revision X, owner sees reliable evidence, identifies uncertainty, directs correction and verifies whether revision Y improved.

Metrics: valid current evidence; citation validity; precision/recall/abstention; time to approved brief; time to verification; useful vs dismissed drift alerts; correction completion; inference cost; repeat active projects; founder/developer task success.

Stop/pivot if users only value prompt generation; if agent APIs cannot stream reliably, do not market real-time monitoring; if drift alerts are noisy, keep them advisory; if founders cannot understand evidence, simplify before adding features; if model cost is high, increase deterministic processing.

17. Research and references

Official references (re-check current versions before coding):

• Codex App Server: https://learn.chatgpt.com/docs/app-server — deep product integration, streamed events, approvals.

• Codex SDK: https://learn.chatgpt.com/docs/codex-sdk — programmatic local agent threads.

• Codex as a platform: https://developers.openai.com/blog/codex-as-a-platform — choose SDK vs App Server based on workflow.

• GitHub Spec Kit: https://github.github.com/spec-kit/ — spec, plan, implement and converge.

• Kiro docs: https://kiro.dev/docs/ — specs, steering, hooks, MCP and permissions.

• Kiro hooks: https://kiro.dev/docs/hooks/ — event-driven automation.

• Kiro steering: https://kiro.dev/docs/steering/ — persistent project context.

• OpenCode docs: https://opencode.ai/docs/ — verify current MCP/plugin/event support.

• Bank of Ghana daily FX rates: https://www.bog.gov.gh/treasury-and-the-markets/daily-interbank-fx-rates/

Existing project source reused: `Build_Brief_MASTER_Blueprint.md` (27 October 2026) from the user's Library. Prior M4e context was a conditional pass: browser flow untested, no live Meta key/call, local-only server and no auth for network exposure, Git commit/ownership checks unresolved. These are historical findings; recheck actual repository before treating them as current.

18. MASTER PROMPT — GIVE THIS TO THE CODING AGENT

You are the principal product engineer and design lead evolving BuildBrief into a reliable, polished, hybrid, cross-agent development control layer. Read this document fully before changing code.

Fixed strategic decisions: BOTH interaction modes (inside BuildBrief and alongside existing agents); HYBRID local-first plus optional cloud; BOTH founders and developers with role-adapted experiences on one shared source of truth.

Mission: build Idea → Clarify → Approve → Build → Observe → Verify → Correct → Understand. Preserve owner-approved versioned intent and connect it to revision-specific evidence. Make uncertainty visible. A prompt is not a patch; source is not runtime proof; AI-extracted requirements are not approved requirements.

First action: inspect actual repository, Git history, dirty state, docs, M4e/M5 reports, architecture, routes, tests, secrets and failures. Do not assume described features already work. Produce inventory of working/partial/broken/untested. Do not delete or rewrite working systems without a migration plan.

Engineering rules

• Preserve and improve evidence-first domain foundation.

• Deterministic local operations before model calls.

• Keep raw intent, AI draft, user decisions and approved intent distinct.

• Immutable intent versions and append-only audit events.

• Evidence pinned to commit/blob, test-run identity and timestamp.

• Mechanically validate every source citation.

• Show evidence grade, search coverage, freshness and capture gaps.

• Say “possibly unrequested change”; require human disposition.

• A correction is unresolved until new revision and fresh checks.

• Provider-neutral adapter contract; never assume all agents support all events.

• Start with alongside mode, CLI/MCP and two adapters; embedded agent via stable SDK/App Server.

• Local-first private source; cloud opt-in and transparent.

• No credentials in source, URLs, prompts or logs.

• Bound and cache context by revision, intent, file hashes, model and prompt version.

• Never fabricate API calls, activity, benchmark scores, runtime proof or integration status.

UX: polished Project Room. Founder mode shows outcomes, decisions and plain English. Developer mode shows criteria, files, diffs, tests and provenance. Same source of truth. Restrained professional visual design, excellent typography, accessible interaction, purposeful motion. No raw enums or internal IDs as primary copy. Do not prioritise 3D over usability.

Build order: (A) audit/stabilise; (B) approval model/evidence freshness; (C) Project Room redesign; (D) adaptive clarification/brief compiler; (E) CLI/MCP and first two adapters; (F) activity timeline; (G) deterministic verification/reverse review; (H) correction/re-check; (I) role explanations; (J) security/cost/tests/docs.

Token discipline: offline fixtures by default; no paid calls for routine tests; one approved live smoke test per new integration; estimate expensive calls; log tokens/retries/latency/cost; stop at cap; do not repeat broad research or test suites without reason.

Separate coding-agent credits from BuildBrief inference.

Quality gate: define acceptance tests before each epic; run targeted tests after each change and full gates at milestones; test negative/adversarial cases; validate UI in browser at desktop/mobile widths; do not claim browser testing unless performed; keep changelog and safe commits; never blind `git add -A` when Git ownership/secrets are uncertain.

Before coding: report repository inventory, risk list, architecture delta, phased task graph, effort/dependencies and genuinely blocking questions. Use reversible defaults for non-blocking unknowns. Then implement incrementally and attach evidence to every acceptance criterion.

Success: a real user describes an idea, answers useful questions, approves a brief, hands it to an agent, sees supported activity, inspects evidence, approves a correction, rechecks a changed revision and understands what happened. Label every simulated step.